Lab 2: NoC Performance Analysis

王立明
2024011338

Abstract

Latency–throughput analysis of an 8×8 Garnet Mesh (XY routing, single-flit packets, 10000 cycles per point). Five synthetic patterns are swept over offered loads 0.01–0.50, followed by single-parameter sweeps of VCs per vnet, router pipeline depth, and link width (the latter repeated with multi-flit packets). Low-load latency follows $T_0 \approx 2h+5$; saturation is set by the most-loaded link, so transpose flattens at an accepted 0.20 packets/node/cycle while uniform random never saturates in range. Four VCs suffice; each extra pipeline stage costs $(h+1)$ cycles; link width matters only for multi-flit packets, where it converts the mesh's constant flit capacity (0.362 flits/node/cycle) into packet throughput. The 1 GHz traffic generators facing the 2 GHz network halve the effective offered load; all ideal lines account for this.

1 Methodology

Every data point is one gem5 run of `garnet_synth_traffic.py` on the Lab-1 build (tick = Ruby cycle, `reception_rate` statistic): 8×8 `Mesh_XY`, 64 CPUs/dirs, `--inj-vnet=0`, `--sim-cycles=10000`. Sweep driver, CSVs and plot scripts accompany this report. Latency is `average_packet_latency` (cycles), throughput is `reception_rate` (packets/node/cycle).

One subtlety governs every plot: the traffic generators tick in the 1 GHz *system* clock domain while Garnet runs at 2 GHz, so an offered rate r (packets/node/CPU-cycle) is $r/2$ per network cycle — visible directly in the data (accepted rate 0.2489 $\approx 0.5/2$ at offered 0.5). Ideal throughput lines therefore have slope 1/2, and saturation is judged against $r/2$. Task-2 sweeps stop at offered 0.40 (multi-flit link-width sweep at 0.30), so a "peak" for an unsaturated configuration is its last measured point.

2 Task 1: Traffic patterns

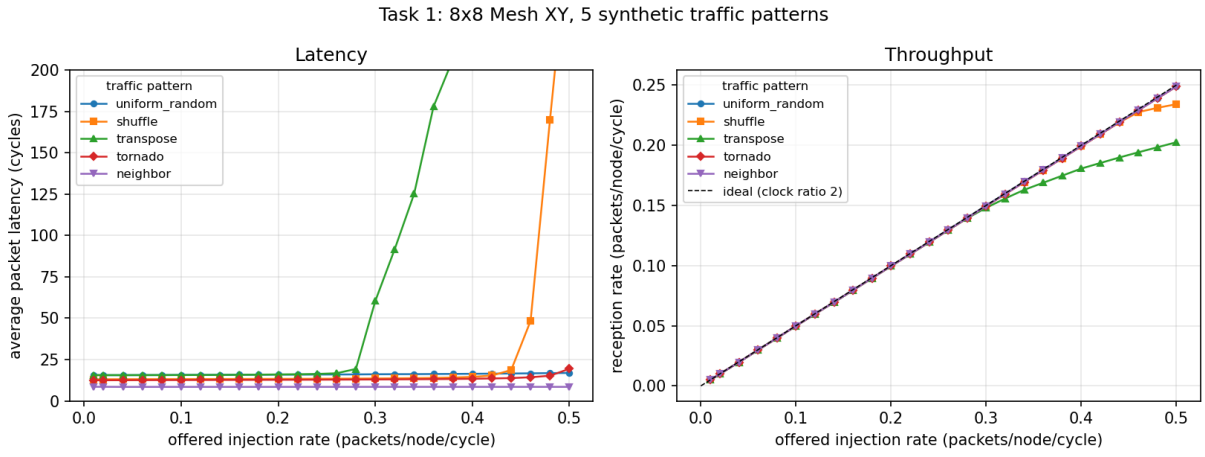

Figure 1: Latency (left) and accepted throughput (right) vs. offered load, five patterns.

Table 1: Key numbers. h: mean hops; T_0: zero-load latency (cycles); sat.: offered rate where the accepted rate first drops below 95 % of ideal; peak: highest accepted rate.

pattern	h	T_0	sat.	peak
uniform_random	5.27	15.56	> 0.5	0.249
shuffle	4.01	13.03	≈ 0.50	0.234
transpose	5.32	15.66	0.36	0.203
tornado	3.73	12.47	> 0.5	0.249
neighbor	1.77	8.53	> 0.5	0.249

Low-load latency is distance. With 1-cycle routers and links, $T_0 \approx 2h+5$ fits all five patterns: one router + one link per hop, plus a constant injection/ejection overhead. (h is router-to-router distance; probe runs show `average_hops` $= 0$ for self-traffic, 1 for a neighbour, 14 corner-to-corner.) Neighbor ($E[h] = 1.75$: the $(x+1)$ mod 8 mapping wraps $x = 7$ onto a 7-hop path) is cheapest; transpose and uniform random share $h \approx 5.3$ and hence T_0.

Throughput is the most-loaded link, not distance. Transpose ($(x, y) \to (y, x)$) is XY's worst case: all 8 packets of row y funnel through the single turn link at (y, y), so its latency knee appears at offered 0.28 and the accepted rate flattens at 0.20. Shuffle is milder (knee ≈ 0.46). Uniform random, tornado and neighbor track the ideal line to the end — with the factor-2 clock ratio their effective load never reaches the mesh's limit. Tornado is the counterexample separating the two effects: a permutation with *short* distance and balanced links behaves like uniform random, so the correlation is with link load balance, not hop count.

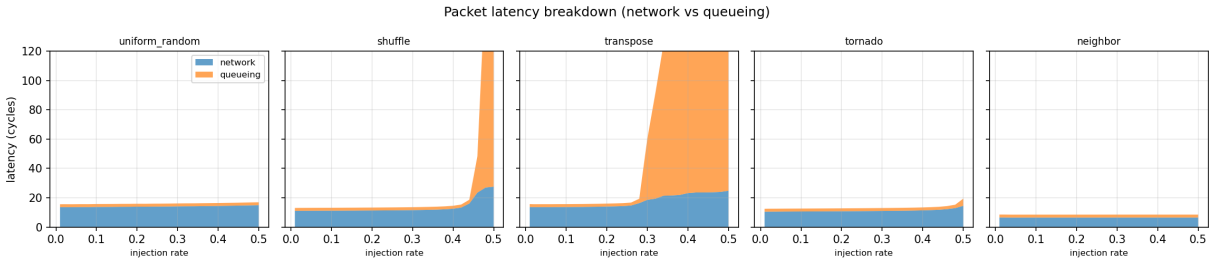

Figure 2: Latency split into in-network vs. source-queueing parts.

Where the latency goes. Past saturation essentially all latency is queueing at the source NI: transpose at offered 0.5 spends 343 of 368 cycles (93 %) in the source queue while the in-network part only grows from 13.6 to 24.8. Below the knee the picture inverts (13.6 network / 2 queueing at 0.01).

3 Task 2: Router parameters

Each knob is swept alone (defaults otherwise: 4 VCs, 1-cycle router, 128-bit links), under uniform random (benign) and transpose (adversarial).

3.1 VCs per vnet

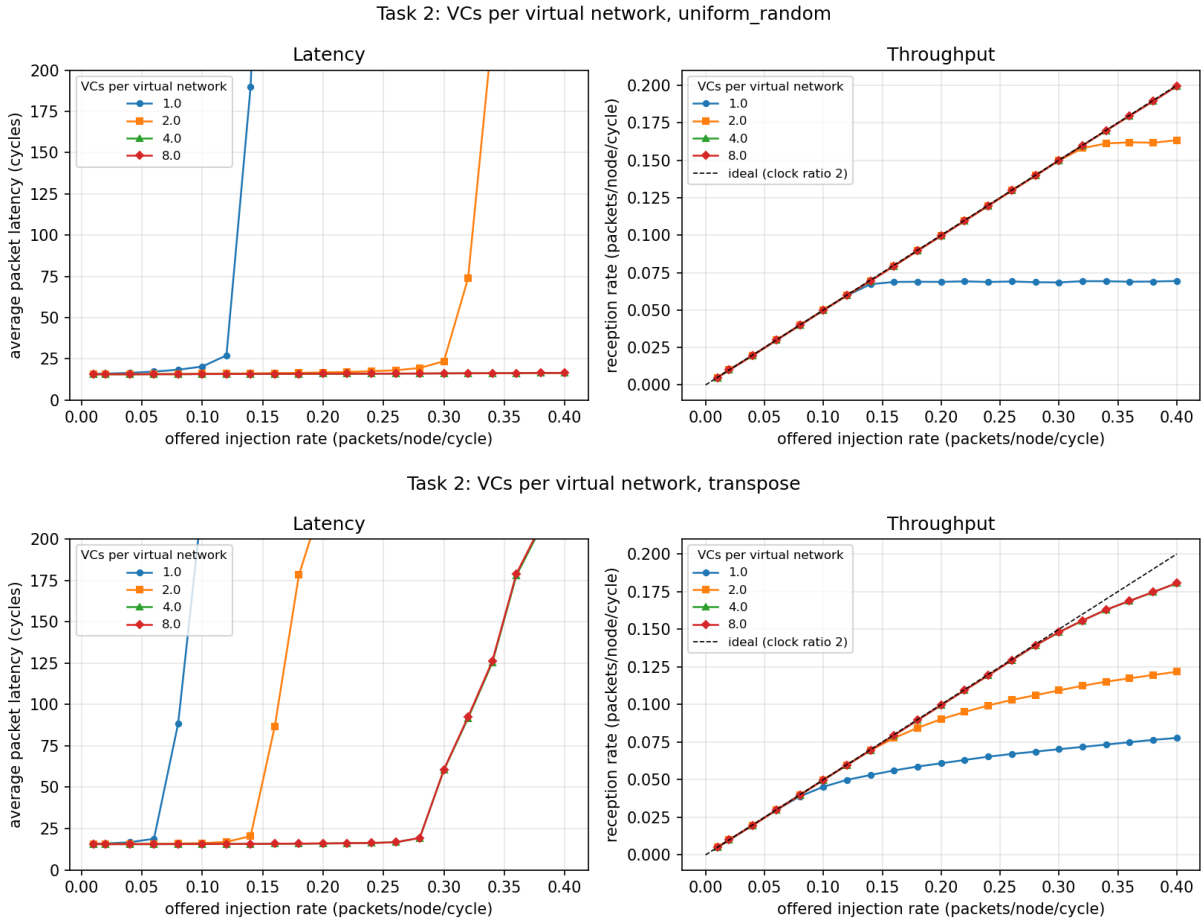

Figure 3: `--vcs-per-vnet` $\in \{1, 2, 4, 8\}$: uniform random (top), transpose (bottom).

Zero-load latency is unchanged — an idle network never waits for a VC. Throughput scales strongly: uniform random accepts 0.069/0.163/0.199 at 1/2/4 VCs. With one VC and a 1-flit buffer, each port holds one packet per credit round trip and unrelated flows block each other head-on. The gain stops at 4 VCs (8 is identical): from there the switch — one flit per output per cycle — is the bottleneck, not VC availability.

3.2 Router pipeline latency

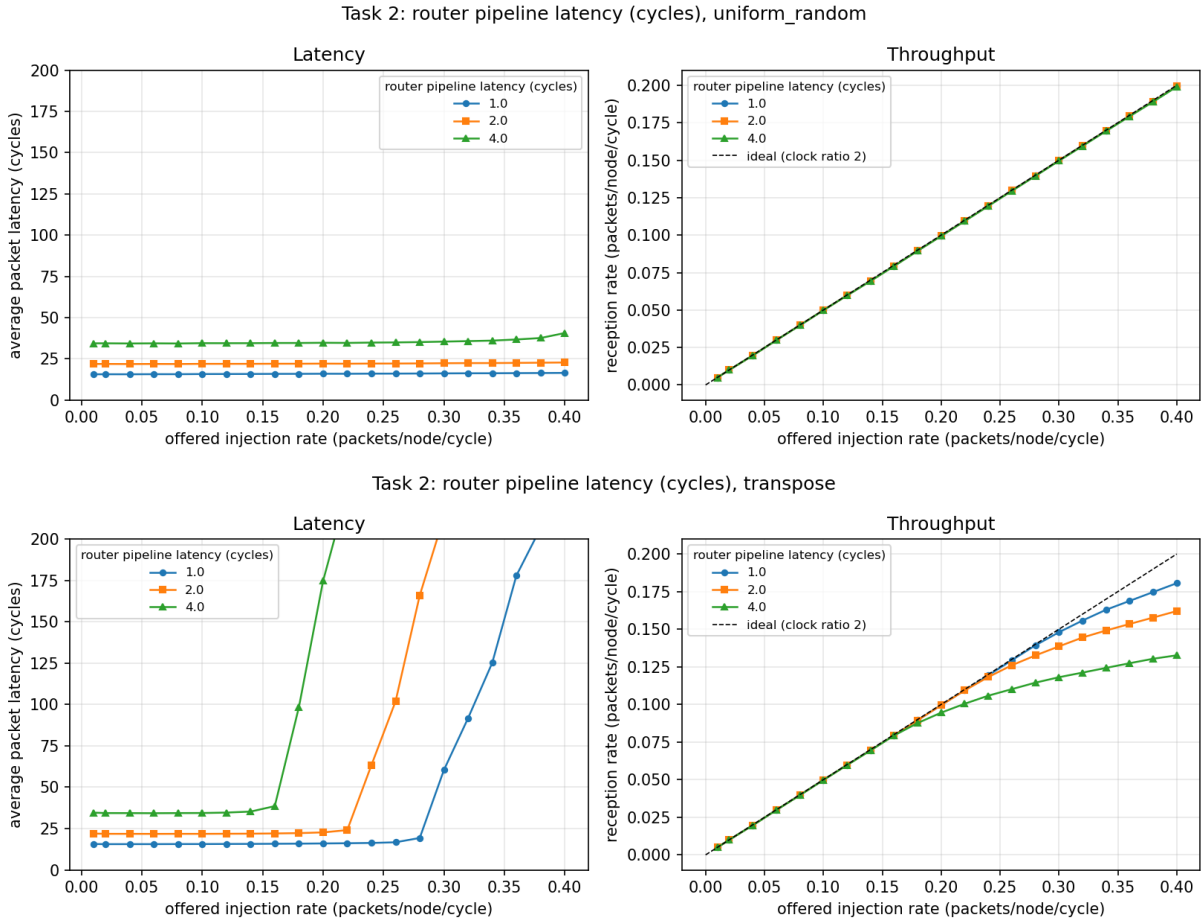

Figure 4: `--router-latency` $\in \{1, 2, 4\}$.

Zero-load latency grows by exactly $(RL-1)(h+1)$: 15.56 → 21.82 → 34.35 for uniform random ($h{+}1 = 6.27$ routers per path). Uniform-random throughput is untouched — pipelining costs latency, not bandwidth — but transpose degrades (0.181 → 0.162 → 0.133): the longer credit round trip is no longer covered by the 1-flit ctrl buffers on its saturated turn links.

3.3 Link width

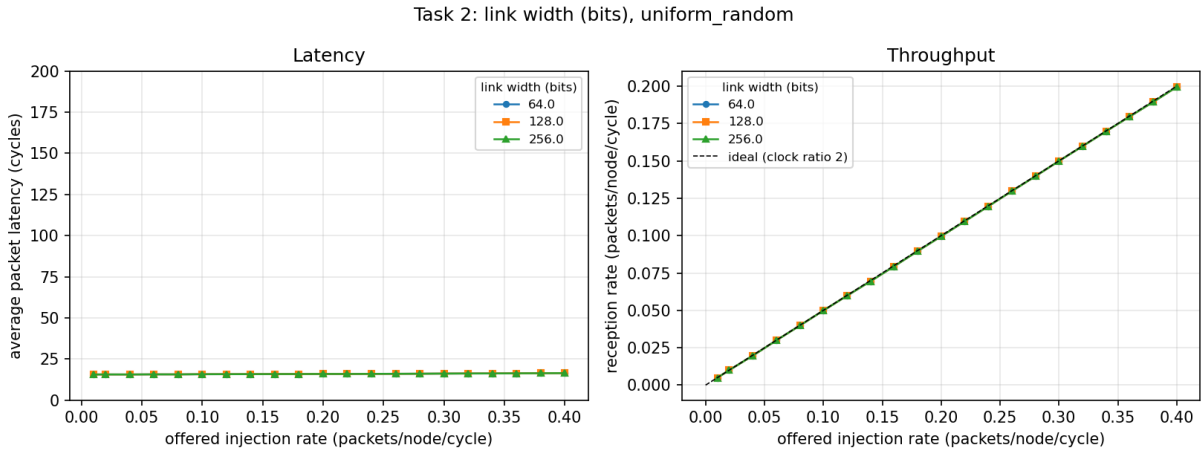

Figure 5: `--link-width-bits` $\in \{64, 128, 256\}$, 1-flit packets: the curves coincide exactly.

On vnet 0 the knob does nothing: Garnet couples flit size to link width (`ni_flit_size = link_width_bits/8`), so the packet is one flit at every width. The sweep was therefore repeated on vnet 2, whose 72-byte packets (64 B data + 8 B header) serialise into 9/5/3 flits at 64/128/256 bits.

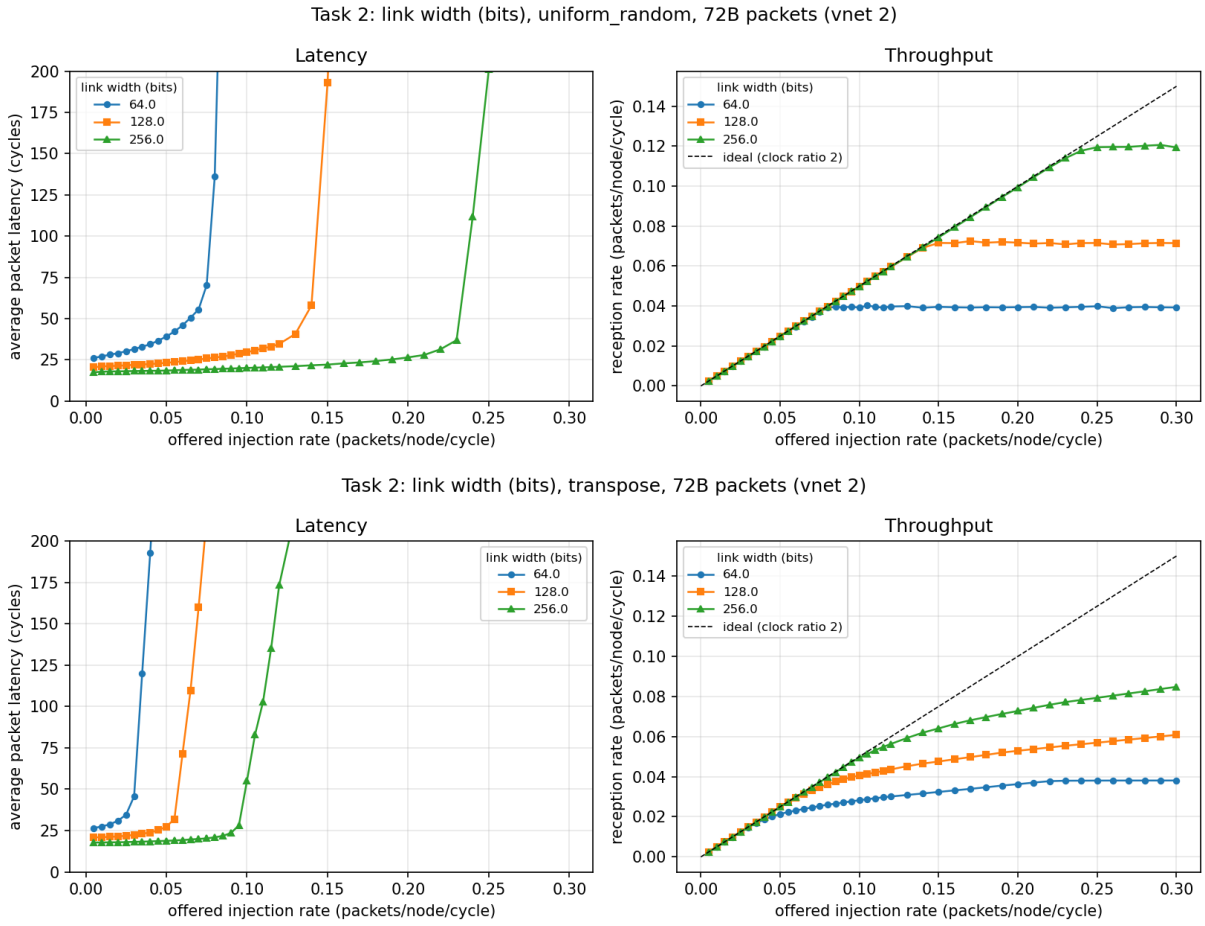

Figure 6: Link width with 72-byte packets (vnet 2): uniform random (top), transpose (bottom).

Now both effects appear. Zero-load latency drops with the serialisation term (26.1 → 20.5 → 17.4 cycles), and the saturated *packet* rate of uniform random scales with width: 0.040 → 0.073 → 0.121 packets/node/cycle. Multiplying by flits/packet gives 0.362 in all three cases — the mesh's *flit* capacity is constant; a wider link just spends fewer flit-cycles per packet. Transpose peaks lower at every width, its turn-link bottleneck wasting part of the unchanged per-link budget.

4 Task 3: Questions

4.1 Components of network latency; dominant terms

$$T = \underbrace{T_{queue}}_{\text{source NI}} + \underbrace{(h{+}1)\,t_{router} + h\,t_{link}}_{\text{pipeline}} + \underbrace{T_{ser}}_{\text{flits/pkt}-1} + \underbrace{T_{cont}}_{\text{contention}}$$

gem5 reports the first term as `average_packet_queueing_latency` and the rest as `average_packet_network_lat`. Which dominates depends on the operating point: at **low load** the deterministic pipeline term ($2h$ here) — hence hop count and router depth are the levers; at **high load / adversarial patterns** source queueing, as soon as the busiest link saturates (transpose: 93 % of latency at offered 0.5); with **narrow links and long packets** the serialisation term ($T_0 = 26.1$ at 64-bit vs 17.4 at 256-bit for 9- vs 3-flit packets).

4.2 Input buffer depth and default flow control

From `GarnetNetwork.py`: `vcs_per_vnet = 4`, `buffers_per_ctrl_vc = 1`, `buffers_per_data_vc = 4`. Each input port thus provides 4 VCs × 1 flit on the two ctrl vnets and 4 VCs × 4 flits on the data vnet; Garnet is purely input-buffered.

The default flow control is **credit-based virtual-channel wormhole flow control with per-packet VC allocation**. Buffer space is allocated per flit — every slot is a credit (`OutVcState::increment/` and `SwitchAllocator::send_allowed()` requires a free output VC (head flits) or a credit on the held VC (body/tail). It is wormhole, not virtual cut-through: a head flit advances as soon as one downstream slot exists — a 5-flit data packet cannot even fit its 4-deep VC, so packets routinely span routers. The VC itself, however, is held per *packet* from head to tail; on the ctrl vnets (1-flit packets, 1-flit buffers) this degenerates into one packet per VC per credit round trip — the restriction Lab 3's `--wormhole` extension removes.

5 Conclusion

Three independent levers govern the curves. Distance and pipeline depth set low-load latency ($T_0 \approx 2h+5$, $+(h{+}1)$ per extra router stage). Traffic placement on links sets the saturation point — transpose collapses far below uniform random at identical mean distance. Buffering decides how close to the link limit the network operates (1 VC throttles every port to its credit round trip; 4 VCs recover full switch bandwidth), and link width converts constant flit capacity into packet throughput only for multi-flit packets. Past saturation, nearly all measured latency is source-side queueing, not time in the network.